SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Brasilla R	**Reg. No.:**	192511357
Date:	28/10/2026	**Duration:**	60 minutes

Code of Conduct

I Brasilla R / 192511357 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: *Brasilla R*

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.
Assignment Questions

1. **Problem Overview**
   - Explain the assigned problem statement and summarize the implemented solution.
   - Describe the project objectives and key functionalities.
2. **Repository Organization**
   - Evaluate the repository structure, folder organization, and file naming conventions.
   - Justify how the repository layout supports maintainability and collaboration.
3. **Code Quality Review**
   - Review the source code for readability, modularity, coding standards, and documentation.
   - Identify strengths and areas for improvement.
4. **Version Control Evaluation**
   - Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
   - Explain how version control supports collaborative software development.
5. **Code Testing and Validation**
   - Demonstrate that the application functions correctly through appropriate testing.
   - Present test cases, execution results, and screenshots as evidence.
6. **Repository Documentation**
   - Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

    - Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
    - Summarize the overall code review findings.
    - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	**Mark**
Problem Analysis & Repository Organization(15%)	/4
Code Quality Review(25%)	/5
Version Control Evaluation(20%)	/4
Testing & Validation(15%)	/4
Documentation & Repository Maintenance(15%)	/4
Review Findings, Recommendations & Presentation(10%)	/4
TOTAL	/25

1. Problem Overview

Assigned problem statement: Develop a system that allows vehicle owners to book service appointments, maintain service history, receive maintenance reminders, and track service status.

Summary of the implemented solution

The Vehicle Service and Maintenance Management System (VSMS) is a three-tier web application. A static page served by Nginx calls a Flask REST API (Python 3.12, Gunicorn), which stores data in PostgreSQL through SQLAlchemy. All three parts run as Docker containers started by one Docker Compose file. The repository under review is named *vsms*, version 1.0.1, and contains about 95 lines of Python (45 in routes.py, 16 in models.py, 15 in the app factory and 19 in tests).

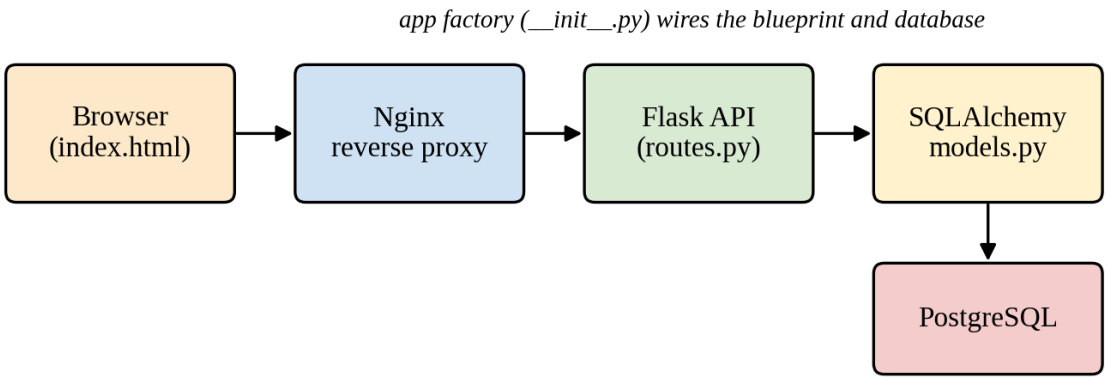


Figure 1: Architecture of the implemented solution

Project objectives

- Allow owners to register vehicles and book service appointments online.
- Keep a permanent service history for each vehicle.
- Calculate the next due date and report vehicles that need service soon.
- Let staff and owners follow the status of a service (Booked, In Progress, Completed).
- Demonstrate good version control, containerization and collaboration practice.

Key functionalities and endpoints

- **POST /api/vehicles** registers a vehicle.
- **POST /api/appointments** books a service.
- **PATCH /api/appointments/<id>/status** updates the status and sets next_due (180 days) when Completed.
- **GET /api/vehicles/<id>/history** lists completed services.
- **GET /api/reminders** lists services due within seven days.
- **GET /api/health** reports that the service is running.

Review approach

The review combined automated tools (flake8 for style, radon for complexity, pytest-cov for coverage), manual reading of every source file, extra negative test cases and an analysis of the Git history.

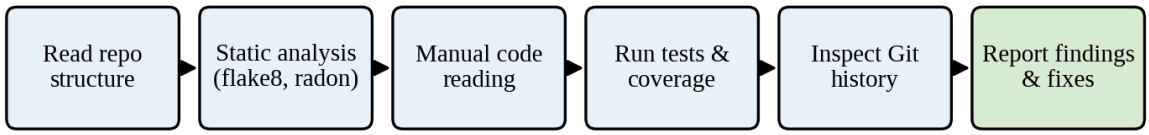


Figure 2: Code review process followed

2. Repository Organization

The files tracked by Git are listed in Figure 3, and the layout is drawn in Figure 4.

```
$ git ls-files
.env.example
.gitignore
README.md
backend/.dockerignore
backend/Dockerfile
backend/app/__init__.py
backend/app/models.py
backend/app/routes.py
backend/requirements.txt
backend/tests/test_api.py
docker-compose.yml
frontend/index.html
frontend/nginx.conf
```

Figure 3: Repository structure (git ls-files)

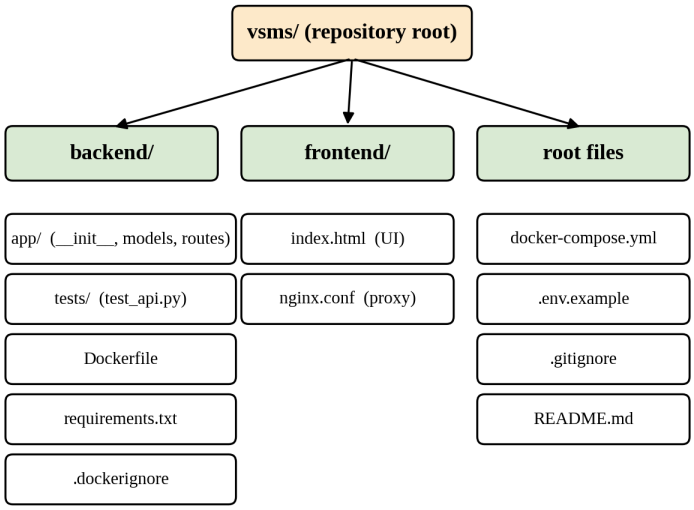


Figure 4: Repository layout diagram

Evaluation

- **Folder organization (Good):** backend, frontend and deployment files are separated, and the application code in backend/app is separate from backend/tests. A new developer can find any file quickly.
- **Naming conventions (Good):** Python files use lowercase snake_case (models.py, routes.py), folders are lowercase, and Docker files use their standard names (Dockerfile, docker-compose.yml, .dockerignore).

- **Hygiene (Good):** .gitignore excludes caches, virtual environments, local databases and the real .env; .env.example documents settings. No compiled files or secrets are tracked.
- **Gaps:** there is no LICENSE, CONTRIBUTING or CHANGELOG file, no docs folder and no CI configuration folder (.github/workflows).

How the layout supports maintainability and collaboration

Because each concern has its own folder, two developers can work on the API and the frontend at the same time without touching the same files, which keeps merge conflicts small. The app factory pattern (create_app) and the blueprint in routes.py allow new modules, such as an owner-login module, to be added as new files. Test code lives beside, not inside, the application, so it is excluded from the Docker image by .dockerignore.

3. Code Quality Review

Readability

Names are meaningful (add_vehicle, book, update_status, reminders, next_due) and functions are short, the longest being about ten lines. Routes follow one consistent style. However, the code contains no docstrings or comments, and two lines combine two statements with a semicolon, which reduces clarity.

```
# routes.py (line 16)
db.session.add(v); db.session.commit()
```

Modularity

Responsibilities are clearly separated: __init__.py builds and configures the application, models.py defines tables, and routes.py handles HTTP. The database URL is read from an environment variable, so no environment-specific value is hard-coded. The remaining weakness is that business logic (the 180-day rule and the 7-day reminder window) lives directly inside route functions and uses unnamed numbers.

Coding standards

Running flake8 with a 100-character limit produced 20 warnings, all on style or tidiness rather than logic errors (Figures 5 and 6). The majority are E302 (two blank lines are needed between top-level definitions).

```
$ flake8 app tests --max-line-length=100
app/__init__.py:7:1: E302 expected 2 blank lines, found 1
app/__init__.py:9:101: E501 line too long (111 > 100 characters)
app/models.py:1:1: F401 'datetime.date' imported but unused
app/models.py:4:1: E302 expected 2 blank lines, found 1
app/models.py:10:1: E302 expected 2 blank lines, found 1
app/routes.py:8:1: E302 expected 2 blank lines, found 1
app/routes.py:12:1: E302 expected 2 blank lines, found 1
app/routes.py:16:22: E702 multiple statements on one line (semicolon)
app/routes.py:19:1: E302 expected 2 blank lines, found 1
app/routes.py:24:22: E702 multiple statements on one line (semicolon)
app/routes.py:27:1: E302 expected 2 blank lines, found 1
app/routes.py:30:101: E501 line too long (108 > 100 characters)
app/routes.py:32:1: E302 expected 2 blank lines, found 1
app/routes.py:41:1: E302 expected 2 blank lines, found 1
app/routes.py:44:101: E501 line too long (105 > 100 characters)
tests/test_api.py:4:1: E302 expected 2 blank lines, found 1
tests/test_api.py:9:1: E302 expected 2 blank lines, found 1
tests/test_api.py:12:1: E302 expected 2 blank lines, found 1
tests/test_api.py:13:101: E501 line too long (127 > 100 characters)
tests/test_api.py:14:101: E501 line too long (126 > 100 characters)
20
```

Figure 5: flake8 output

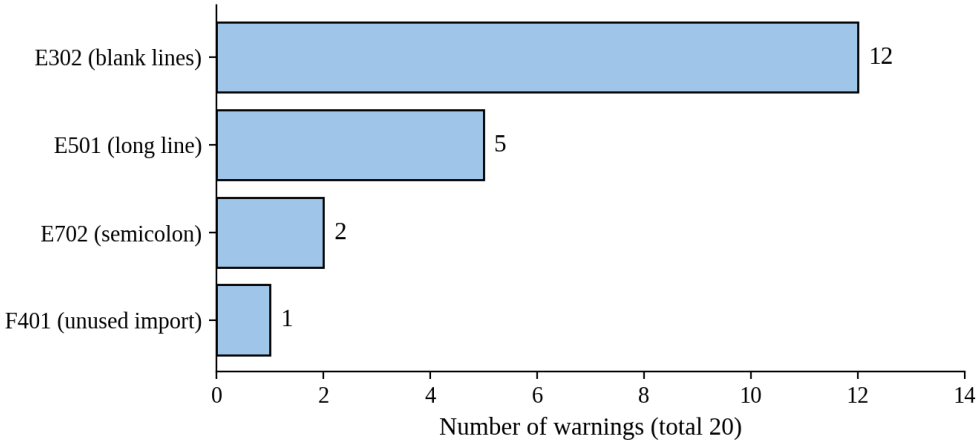
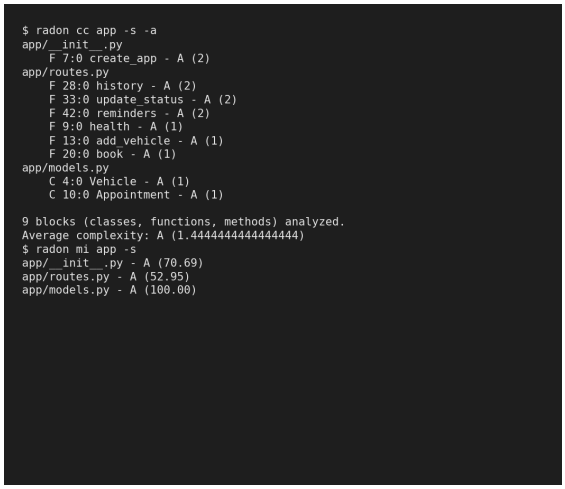


Figure 6: flake8 warnings by type

Complexity and maintainability metrics

Radon reports every function at grade A with an average cyclomatic complexity of 1.44, and a maintainability index of 100, 71 and 53 for models.py, __init__.py and routes.py respectively, all in the A band. The code is therefore easy to understand and to test.



```
$ radon cc app -s -a
app/__init__.py
    F 7:0 create_app - A (2)
app/routes.py
    F 28:0 history - A (2)
    F 33:0 update_status - A (2)
    F 42:0 reminders - A (2)
    F 9:0 health - A (1)
    F 13:0 add_vehicle - A (1)
    F 20:0 book - A (1)
app/models.py
    C 4:0 Vehicle - A (1)
    C 10:0 Appointment - A (1)

9 blocks (classes, functions, methods) analyzed.
Average complexity: A (1.4444444444444444)
$ radon mi app -s
app/__init__.py - A (70.69)
app/routes.py - A (52.95)
app/models.py - A (100.00)
```

Figure 7: Complexity and maintainability index (radon)

Strengths and areas for improvement

Strengths	Areas for improvement
Clear separation of app factory, models and routes	No docstrings or inline comments
Very low complexity (average 1.44)	20 PEP 8 style warnings, one unused import (datetime.date in models.py)
Configuration through environment variables	Request data is read with d["key"], so a missing field causes HTTP 500
Non-root container user and health checks	Status accepts any text, not only Booked, In Progress or Completed
Consistent REST-style URLs under /api	Magic numbers (180 days, 7 days) are not named constants

Example of an issue and its fix

```
# Current code
d = request.get_json()
v = Vehicle(owner=d["owner"], reg_no=d["reg_no"], model=d["model"])

# Recommended
d = request.get_json(silent=True) or {}
missing = [k for k in ("owner", "reg_no", "model") if k not in d]
if missing:
    return jsonify(error="missing: " + ", ".join(missing)), 400
```

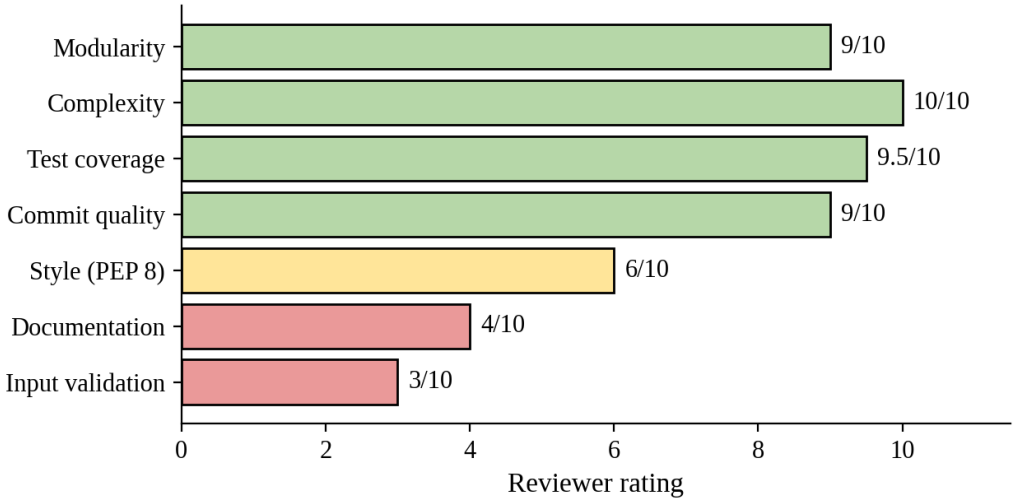


Figure 8: Reviewer scorecard

4. Version Control Evaluation

Commit history

The develop branch contains 15 commits made by one author, and the repository holds two annotated release tags (v1.0.0 and v1.0.1) and eight branches. The history graph is shown in Figure 9 and the individual commits in Figure 10.

```
$ git log --oneline --graph --all --decorate
*   0e47596 (tag: v1.0.1, origin/main, main) hotfix: reminder null check
|\
| * 23ba84a (HEAD -> develop, origin/hotfix/reminder-null-check, origin/develop, hotfix/reminder-null-check) fix(api): guard reminders query against null next_due dates
|/
*   77ed8c8 (tag: v1.0.0) release: v1.0.0
|\
| * 4be6f36 (origin/release/v1.0.0, release/v1.0.0) docs: update README for v1.0.0 release
| *   24d9b07 merge: feature/dockerization into develop
| |\
| | * 4c8bbf0 (origin/feature/dockerization, feature/dockerization) build(docker): add docker-compose with db, backend and frontend services
| | * 249b78b build(docker): add backend Dockerfile, dockerignore, nginx frontend config
| |/
| * 248a8bd test(api): add pytest cases for health and booking-to-history flow
| *   e2deb4d merge: feature/status-reminders into develop (resolve routes.py conflict)
| |\
| | * d673f18 (origin/feature/status-reminders, feature/status-reminders) feat(api): add status tracking and maintenance reminder endpoints
| * |   d06dce1 merge: feature/service-history into develop
| |\ \
| | |/
| |/|
| | * fc0bfa2 (origin/feature/service-history, feature/service-history) feat(api): add service history endpoint
| |/
| *   f57d514 merge: feature/appointment-booking into develop
| |\
| | * c3a03ce (origin/feature/appointment-booking, feature/appointment-booking) feat(api): add vehicle registration and appointment booking endpoints
| |/
| * b476d52 feat(backend): add Flask app factory and Vehicle/Appointment models
|/
* 92410e7 chore: initialize repository with README, .gitignore and .env.example
```

Figure 9: Commit graph of all branches

```
$ git log --oneline --no-merges
23ba84a fix(api): guard reminders query against null next_due dates
4be6f36 docs: update README for v1.0.0 release
248a8bd test(api): add pytest cases for health and booking-to-history flow
4c8bbf0 build(docker): add docker-compose with db, backend and frontend services
249b78b build(docker): add backend Dockerfile, dockerignore, nginx frontend config
d673f18 feat(api): add status tracking and maintenance reminder endpoints
fc0bfa2 feat(api): add service history endpoint
b476d52 feat(backend): add Flask app factory and Vehicle/Appointment models
c3a03ce feat(api): add vehicle registration and appointment booking endpoints
92410e7 chore: initialize repository with README, .gitignore and .env.example

$ git shortlog -sn --all
    16 Brasilla R
```

Figure 10: Commit list (merges excluded) and author summary

Commit messages

Every message follows the Conventional Commits style (type, optional scope, short imperative description). Figure 11 shows the commit types used. Messages describe what changed, for example "fix(api): guard reminders query against null next_due dates", which makes the log useful as a changelog. Commits are small and single-purpose. A possible improvement is adding a message body for the larger commits to explain why the change was made.

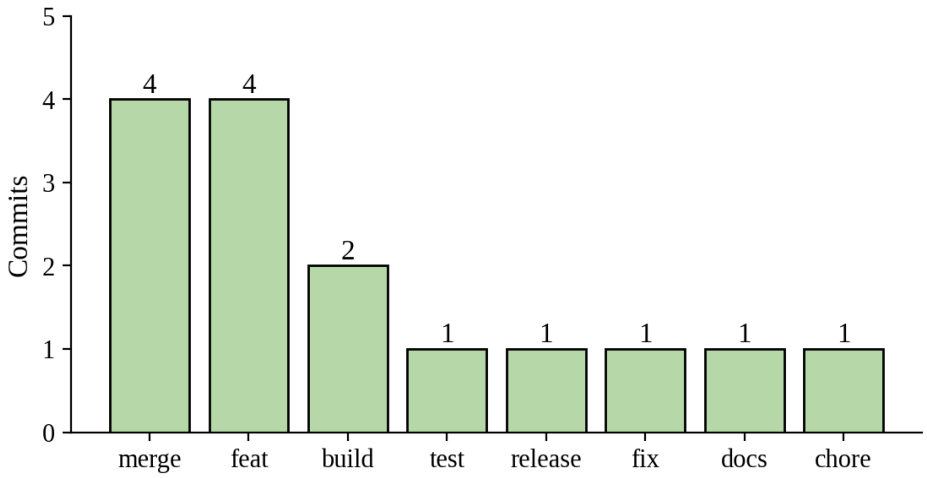


Figure 11: Commits by type

Branching strategy

The repository follows Git Flow: main holds tagged releases, develop integrates features, feature branches carry individual requirements, a release branch prepared v1.0.0 and a hotfix branch corrected the reminders query for v1.0.1. Merges use --no-ff so each feature remains visible in history. A real merge conflict in routes.py was resolved and committed with a clear message.

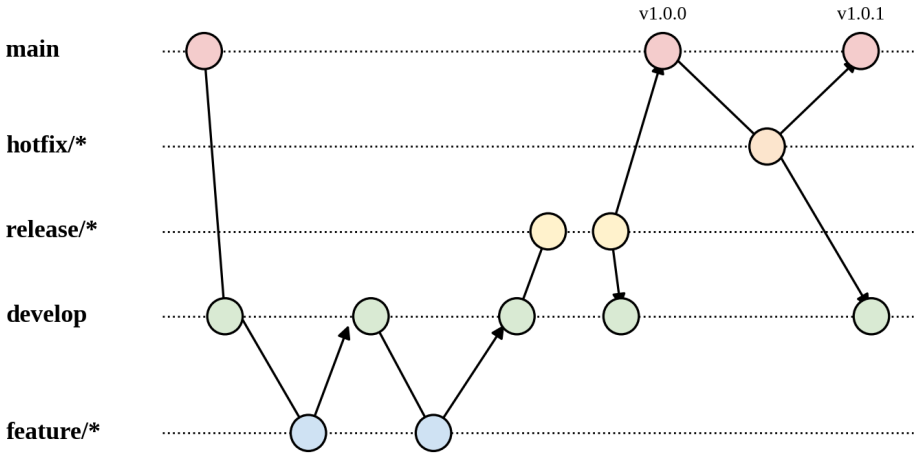


Figure 12: Branching model used in the repository

Repository maintenance practices

- **Good:** the remote origin holds all branches and tags, the working tree is clean, and secrets are ignored.
- **To improve:** delete merged feature branches, protect main and develop, add a pull-request template and require at least one review.

How version control supports collaborative development

Git lets several developers work in parallel on separate branches without overwriting each other. Pull requests enable peer review before code reaches develop, the history shows who changed what and why, any faulty change can be reverted, and tags allow a specific release to be rebuilt or rolled back.

5. Code Testing and Validation

Testing was carried out at two levels: the two automated tests already in the repository, and four additional test cases written during the review to probe reminders, error handling and input validation. All tests use an in-memory SQLite database, so they run without Docker.

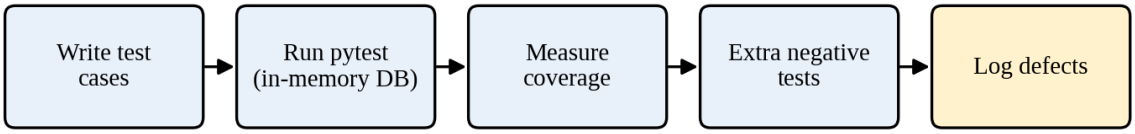


Figure 13: Testing procedure

Test cases and results

ID	Test case	Expected	Result
TC1	GET /api/health	200, status ok	Pass
TC2	Register vehicle, book, complete, read history	201, Booked, Completed, 1 history row	Pass
TC3	Reminder for service completed 175 days ago	1 reminder listed	Pass
TC4	Update status of unknown appointment 999	404	Pass
TC5	Create vehicle without reg_no	400	Fail (500)
TC6	Set status to "Banana"	400	Fail (200)

Execution evidence

```
$ cd backend && pytest -v
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/vsms/backend
collecting ... collected 2 items

tests/test_api.py::test_health PASSED                                    [ 50%]
tests/test_api.py::test_booking_flow PASSED                              [100%]

============================== 2 passed in 0.48s ===============================
```

Figure 14: Existing automated tests (TC1 and TC2)

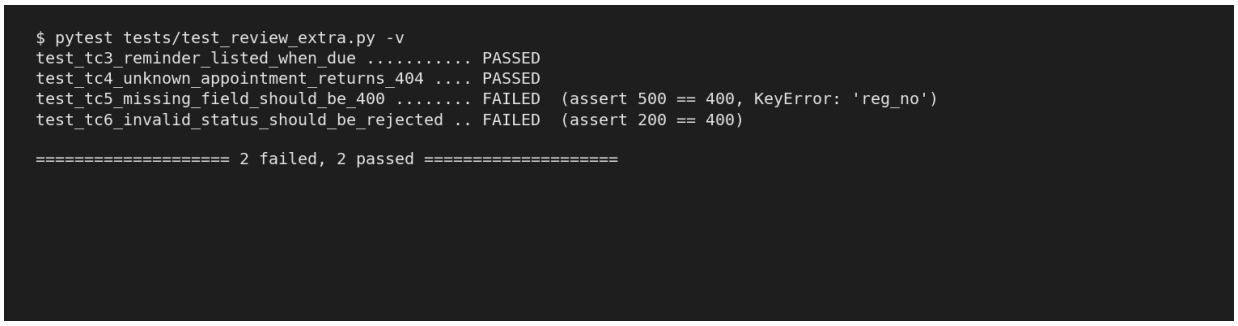


Figure 15: Additional review test cases (TC3 to TC6)

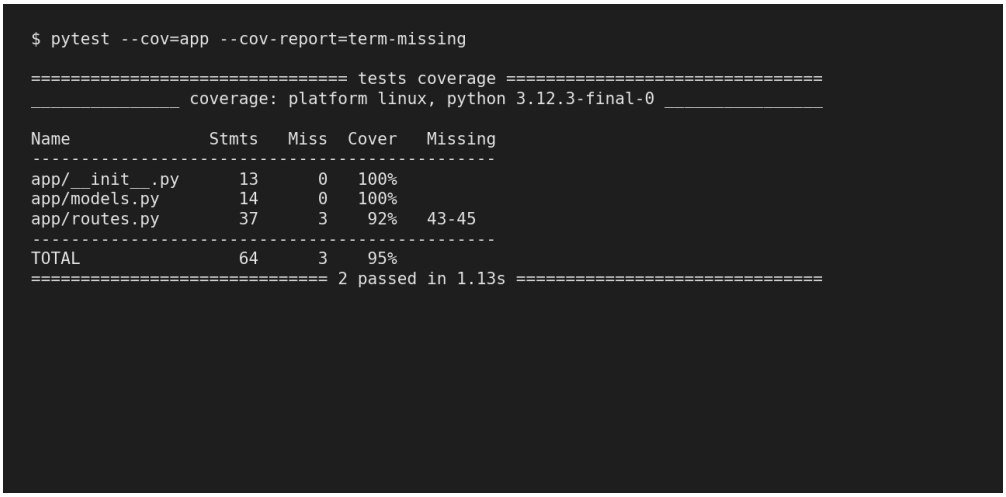


Figure 16: Test coverage report

Analysis

The existing tests pass and give 95 percent statement coverage, but they only cover the successful path. The additional tests confirmed that the reminder logic works (TC3) and that unknown identifiers are handled correctly (TC4). They also exposed two real defects: missing input crashes the API with a server error (TC5) and invalid status values are stored (TC6). These should be fixed and the failing tests then added permanently to the test suite.

6. Repository Documentation

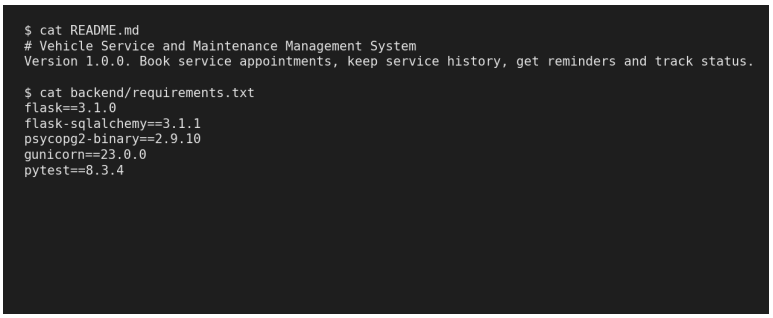


Figure 17: Current README and dependency file

Evaluation

Item	Status	Comment
README overview	Partial	Two lines describing the project only
Installation guide	Missing	No steps for Docker or local setup
Usage guide	Missing	Endpoints and example requests not documented
Dependencies	Good	Pinned versions in requirements.txt
Configuration	Good	Documented by .env.example
Contributing, licence	Missing	No CONTRIBUTING or LICENSE file

Suggested improvements

- Expand the README with a feature list, architecture diagram, prerequisites, clone and run commands (cp .env.example .env, docker compose up --build), the URL to open and how to run tests.
- Add an API reference table with example curl requests and responses.
- Add badges for build and test status, a LICENSE file and a CONTRIBUTING guide explaining the branching model and commit message format.
- Add docstrings to each route so that API documentation can be generated.
- Keep a CHANGELOG.md updated from the release tags.

7. Code Review Findings and Recommendations

Summary of findings

- **Overall:** the project is well structured, simple and maintainable, with excellent version control habits and good test coverage of the main flow.
- **Main defects:** missing input validation (HTTP 500), no status whitelist, no authentication.
- **Quality gaps:** 20 style warnings, no docstrings, documentation too short, limited negative testing.

Recommendations

Area	Recommendation	Priority
Code quality	Validate input and return 400; whitelist the status values; replace magic numbers with constants; run black and flake8; remove unused import	High
Repository management	Protect main and develop, require pull requests, delete merged branches, add pre-commit hooks	Medium
Documentation	Complete README, API reference, LICENSE, CONTRIBUTING	High
Testing	Add TC5 and TC6 as permanent tests, test every error path, raise coverage with a CI gate	High
Future enhancements	Authentication and roles, e-mail or SMS reminders, pagination, database migrations, HTTPS	Medium

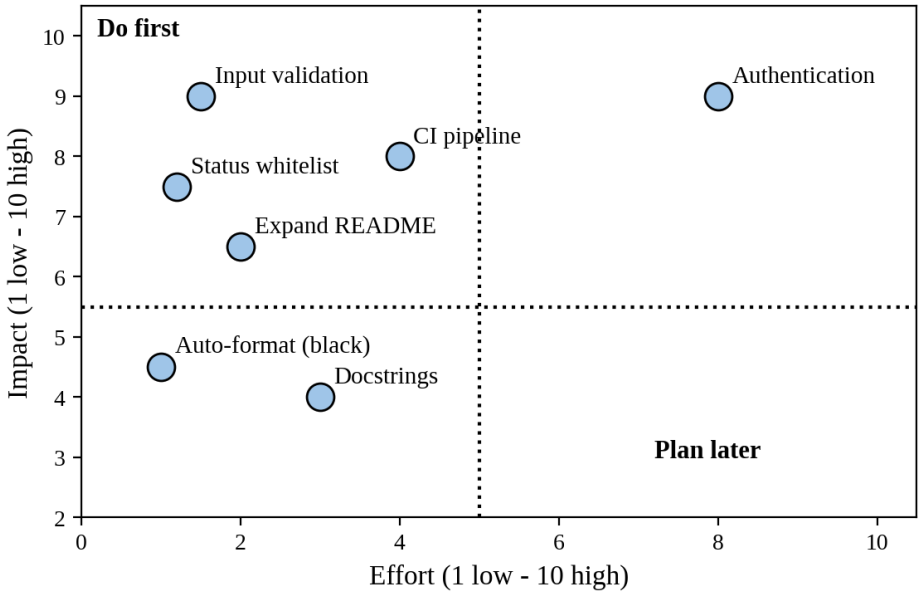


Figure 18: Impact and effort of the recommendations

Applying the high-priority items first gives the greatest improvement for the least effort and would raise the project from a good prototype to a dependable production-ready service.

References

- PEP 8 - Style Guide for Python Code. https://peps.python.org/pep-0008/
- Flake8 Documentation. https://flake8.pycqa.org
- Radon Documentation. https://radon.readthedocs.io
- Pytest Documentation. https://docs.pytest.org
- Chacon, S. and Straub, B., *Pro Git*, 2nd ed. https://git-scm.com/book
- Conventional Commits. https://www.conventionalcommits.org
- Docker Documentation. https://docs.docker.com